

Word Count in MapReduce

Mahmoud Parsian

Ph.D. in Computer Science

Where the Full Mechanics Already Live

The step-by-step Word Count mapper/reducer/combiner/filter mechanics — input splitting, mapper output, sort & shuffle, reducer output, filters applied at the mapper vs. the reducer — are already worked out in detail in three companion documents:

- [mapreduce_examples/MapReduce_Word_Count.md](#) — the classic version, with filters
- [word_count_in_mapreduce/word_count_in_mapreduce.md](#) — most detailed, ends with a PySpark preview
- [combiners/Word_Count_in_MapReduce.md](#) — with vs. without a combiner, side by side

This document instead covers what those three **don't**: running Word Count *at scale*, in an actual cluster.

Quick Recap

Same core idea as

[02_introduction_to_mapreduce.md](#)'s

worked example: `map()` tokenizes each record into `(word, 1)` pairs, Sort & Shuffle groups them by word, `reduce()` sums each word's group into a final count.

```
map(key, value):  
"fox jumped and jumped" ->  
(fox,1) (jumped,1) (and,1) (jumped,1)
```

```
shuffle:  
(fox,[1]) (jumped,[1,1]) (and,[1])
```

```
reduce(key, values):  
(fox,1) (jumped,2) (and,1)
```

How Many Mappers Are Needed?

The answer depends on the number of partitions **and** how many mappers your cluster can actually run at once.

Example: 100,000,000,000 records, partitioned into 50,000 chunks (2,000,000 records each). The *optimal* number of mappers is 50,000, all running in parallel — but what if the cluster only has 10,000?

Answer: the cluster manager assigns one partition to each of the 10,000 mappers; as each mapper finishes, it's handed another partition, until all 50,000 partitions are processed. (The same iterative-assignment idea from

[01_understanding_parallelism_and_concurrency.md](#)'s

"1000 mappers, 100,000 partitions" example.)

How Many Reducers Are Needed?

Sort & Shuffle produces N unique keys, each with its own (possibly very different) number of values:

```
(key_1, [v_11, v_12, ...])  
(key_2, [v_21, v_22, ...])  
...  
(key_N, [v_N1, v_N2, ...])
```

Optimal: N reducers, running in parallel. **If the cluster is smaller** — say $N = 7,000$ unique keys but only 500 reducers available — the cluster manager assigns 500 reduce operations at a time, handing out the next one as each reducer finishes, until all 7,000 are done. Same iterative-assignment pattern as mappers, above.

The Partitioner: Which Key Goes to Which Reducer?

The **partitioner** decides which (key, value) pairs are routed to which reducer. Default:

```
partition = key.hashCode() % numReducers
```

You can override this default when you need:

- **More uniform load** across reducers than the default hash gives you.
- **Co-location** — some keys *must* land on the same reducer. E.g. computing the relative frequency of a word pair <w1, w2> requires every record involving w1 to reach the *same* reducer, regardless of what w2 is.

Co-location, With Numbers: The Problem

Partitioning key `"fox, jumped"` and key `"fox, over"` **by the whole string** can send them to *different* reducers — breaking any reducer logic that needs to see every `fox` pair together (e.g. "what fraction of `fox` pairs are `fox, jumped` ?"):

```
partition(key)          = hash(key)          % n    <- WRONG for this case
partition("fox, jumped") -> Reducer 2
partition("fox, over")   -> Reducer 0  (!)
```

Co-location, With Numbers: The Fix

Partitioning by `w1` alone fixes it — every `fox,*` pair now lands on the same reducer, regardless of `w2` :

```
partition(key)          = hash(key.split(",")[0]) % n
partition("fox,jumped") = hash("fox") % n           -> Reducer 1
partition("fox,over")   = hash("fox") % n           -> Reducer 1 ✓
```

Running in a Cluster: Master and Workers

Cluster = {1 Master node, N worker nodes}

- The **Master** manages the whole cluster and schedules work — it does little computation itself; mappers and reducers run on the **N** worker nodes.
- Scheduling tries to place computation **close to the data** (bandwidth is expensive and slow) — this relies on the underlying distributed file system (GFS/HDFS; see [06_introduction_to_hdfs.md](#)).
- If a worker fails, its tasks go to another worker; the Master itself is handled entirely by the framework — no user code needed.

Failure Handling and Speculative Execution

- A task that stops reporting progress (or whose machine goes down) is assumed stuck, killed, and **re-launched with the same input** — safe because nodes are deterministic and side-effect-free.
- If a straggling task is the *last* one left and is completing slowly, the Master can launch a **second copy** of it elsewhere — whichever copy finishes first wins, and the other is killed.

This is the same fault-tolerance story as

[02_introduction_to_mapreduce.md](#)

("Failure Is the Norm"), one level more concrete.

Two More Job Components

Output Committer — takes the reducer's output and commits it to a file; typically pairs with a matching input splitter so a *downstream* job can read that output. The built-in committer is usually enough unless you need an unusual output file format.

Writables (a Hadoop-specific concept) — types that can be serialized/deserialized to a stream, required for anything crossing the mapper/reducer boundary or hitting disk. A typical job needs (at least) **six**: 2 for input, 2 for intermediate (map → reduce) values, 2 for output. Defaults exist for strings, integers, longs, etc.; you can implement the interface for your own types.

Chaining Multiple Jobs

A single MapReduce job is one `map()` + one `reduce()` — MapReduce itself has no built-in notion of "then run *another* job on this output." Real pipelines often need exactly that (Word Count's own "That's it!" driver class is only simple because it's a single job).

Two common ways to express a multi-step workflow as a graph of jobs:

- **Hadoop YARN** — a Directed Acyclic Graph (DAG) of job nodes
- **Apache Oozie** — a workflow scheduler that also expresses jobs as a DAG

Chaining Multiple Jobs: A Concrete Example

A 2-stage pipeline built entirely from jobs already in this series:

```
Job 1 (04_word_count_in_mapreduce.md):  
  input.txt -> word counts -> counts.tsv
```

```
Job 2 (05_filters_in_mapreduce.md, reducer-side filter):  
  counts.tsv -> keep only (word, count) where count >= 100 -> top_words.tsv
```

Job 2's mapper reads `counts.tsv` as its input — `(word, count)` pairs, not raw text — and its reducer applies the `count >= 100` filter before emitting. Neither job needs to know the other exists; the DAG (YARN or Oozie) just runs Job 2 after Job 1 completes.

Where This Fits

- **Combiners** for Word Count (with vs. without, side by side):
[07_combiners_in_mapreduce.md](#) and
[combiners/Word_Count_in_MapReduce.md](#)
- **HDFS**, the storage layer underneath the Master/worker architecture above: [06_introduction_to_hdfs.md](#)
- **Filtering** at the mapper vs. the reducer:
[05_filters_in_mapreduce.md](#)

Conclusion

MapReduce provides a simple way to scale a big data application: **scale-out**, not scale-up — effortlessly growing from a single machine to thousands, fault-tolerant and high-performance. If your use case fits the paradigm, scaling is handled by the framework, not by you.